

StringeeX Django — Extended Developer Notes (Part 2)

Companion to: StringeeX_Developer_Notes.md

These notes do not repeat Part 1. Read both.

Table of Contents

1. [CSS Design System & Variables](#)
 2. [Animation System](#)
 3. [HTML Structure Map](#)
 4. [Frontend Patterns — Deep Dive](#)
 5. [SheetJS Export System](#)
 6. [Phone Number Masking](#)
 7. [Heatmap Rendering Logic](#)
 8. [KPI Animation System](#)
 9. [Django Patterns Used in This Project](#)
 10. [PostgreSQL Specifics](#)
 11. [Extending the Excel Lookup — Advanced Scenarios](#)
 12. [Adding New Filter Dimensions](#)
 13. [Building a New Dashboard Page](#)
 14. [Security Model — Full Picture](#)
 15. [StringeeX API — Full HTTP Contract](#)
 16. [Performance & Scaling](#)
 17. [Testing Patterns](#)
 18. [Reusable Recipes for Similar Projects](#)
-

1. CSS Design System & Variables

The entire dashboard is styled with a single CSS `:root` block. Understanding this is essential before touching any visual element.

CSS

```
:root {
  /* Background layers – always use these in order, never arbitrary hex */
  --bg: #050810; /* page background – darkest */
  --bg2: #080d1a; /* card/sidebar background */
  --bg3: #0c1220; /* input/hover background */
  --bg4: #101828; /* total column, active elements */

  /* Borders – use these, not arbitrary rgba */
  --border: #1a2540; /* default border */
  --border2: #243050; /* emphasis border, total column */

  /* Text hierarchy */
  --dim: #3d5080; /* barely visible – labels, placeholders */
  --muted: #6b82a8; /* secondary info */
  --text: #c4d4f0; /* body text */
  --bright: #e8f0ff; /* headings, important values */

  /* Accent system – each has a "glow" counterpart */
  --coral: #ff5f6d; /* high activity, danger, outbound calls */
  --coral2: #ff8a5c; /* gradient pair for coral */
  --coral-glow: rgba(255,95,109,.18);

  --cyan: #00d4ff; /* primary accent, connected state, cyan elements */
  --cyan2: #00ffcc; /* gradient pair for cyan */
  --cyan-glow: rgba(0,212,255,.15);

  --gold: #ffd166; /* talk time, warnings, medium activity */
  --gold-glow: rgba(255,209,102,.12);

  --violet: #b57aff; /* TL badges, total duration */
  --green: #00e5a0; /* connected calls, talk duration, success */

  /* Fonts */
  --font: 'Space Grotesk', sans-serif; /* body text, UI elements */
  --mono: 'JetBrains Mono', monospace; /* numbers, codes, labels */
  --display: 'Bebas Neue', sans-serif; /* KPI large numbers only */

  --r: 10px; /* border-radius default */
}
```

Colour Semantics (Never Break These)

Colour	Always Used For
<code>--cyan</code>	Active states, filters applied, connected state, primary interactive
<code>--coral</code>	High volume, outbound direction, danger/high-activity heat
<code>--gold</code>	Medium activity heat, time metrics (talk duration)
<code>--green</code>	Success/connected calls, talk time, positive outcomes
<code>--violet</code>	TL badge, total duration (purple = "accumulated")
<code>--dim</code>	Labels, column headers, placeholder text — never use for values
<code>--bright</code>	Call counts, headings, anything the user must READ

When to Use Each Font

css

```
font-family: var(--display); /* Bebas Neue — KPI big numbers ONLY */
font-family: var(--mono);    /* JetBrains Mono — all timestamps, codes, counts, labels
font-family: var(--font);    /* Space Grotesk — agent names, buttons, prose */
```

A common mistake: using `--font` for a number shown in a table cell. Numbers should be `--mono` because monospace makes columns align.

The `--kpi-accent` CSS Custom Property Pattern

Each KPI card sets its own accent via inline style:

html

```
<div class="kpi-card" style="--kpi-accent:var(--cyan)">
```

The `.kpi-card::before` pseudo-element (animated top border on hover) reads `var(--kpi-accent, var(--cyan))`. This is a CSS custom property with a fallback — a clean pattern for per-instance theming without JavaScript.

To add a new KPI card with a different accent:

html

```
<div class="kpi-card" style="--kpi-accent:var(--coral)">
  <div class="kpi-label">My Metric</div>
  <div class="kpi-value c-coral" id="kpi-mine">—</div>
  <div class="kpi-sub">subtitle</div>
  <div class="kpi-bg-icon">🏠</div>
</div>
```

2. Animation System

All animations are CSS keyframes, applied via class or `:nth-child`. No JS animation libraries.

Keyframe Inventory

CSS

```
@keyframes slide-in-left /* sidebar entrance - translateX(-16px) → 0 */
@keyframes slide-in-up /* KPI cards, table rows, cards - translateY(12px) → 0 */
@keyframes fade-in /* simple opacity 0 → 1 */
@keyframes count-up /* KPI values - scale(.85) + opacity 0 → 1 */
@keyframes bar-fill /* heat bar - width 0 → set value */
@keyframes spin /* loading spinner - rotate 360deg */
@keyframes glow-pulse /* live dot - box-shadow oscillates */
@keyframes pulse-ring /* live dot ring - scale 1→2.5, opacity 1→0 */
@keyframes rotate-slow /* unused, available */
```

Staggered Card Animations

Cards and KPI items use `:nth-child` + `animation-delay` for a cascade effect:

CSS

```
.kpi-card:nth-child(1) { animation: slide-in-up .35s .05s ease both; }
.kpi-card:nth-child(2) { animation: slide-in-up .35s .10s ease both; }
/* ... each .05s later */

.summary-card:nth-child(1) { animation-delay: .03s; }
.summary-card:nth-child(2) { animation-delay: .06s; }
/* ... each .03s later, up to 10 */
```

`animation-fill-mode: both` (the `both` keyword) means the element starts in the `from` state before the animation begins — critical for staggered reveals where elements would otherwise

flash visible before their delay.

To extend stagger beyond 10 cards, add more `:nth-child` rules. Current limit is 10 — if an 11th card appears, it has no delay and pops in instantly.

The `bar-fill` Keyframe Subtlety

CSS

```
@keyframes bar-fill {  
  from { width: 0 !important; }  
  to   { /* width set inline */ }  
}
```

`to` is empty because the final width is set inline on the element. The `!important` in `from` overrides the inline style during the initial frame. Without `!important`, the animation would start at the inline width (no visual effect). This is an unusual but valid CSS pattern.

Live Dot Double-Ring Effect

CSS

```
.live-dot { animation: glow-pulse 2s ease-in-out infinite; }  
.live-dot::after {  
  /* same shape as parent */  
  animation: pulse-ring 1.6s ease-out infinite;  
}
```

The `::after` is a same-size pseudo-element that scales outward and fades — creates the "radar ping" effect. Different timing (1.6s vs 2s) keeps them from syncing.

3. HTML Structure Map

<code><header class="topbar"></code>	– sticky header; holds logo, live badge, countdown, refresh
<code><div class="shell"></code>	– flex row; full viewport minus topbar height
<code><aside class="sidebar"></code>	– 252px fixed; filters + agent list
<code>.sidebar-head</code>	– icon + "Team Filters" label
<code>.filter-block × 4</code>	– Date, TL, Pool, Search Agent
<code>.team-list</code>	– scrollable agent list; re-rendered on each breakdown load
<code></aside></code>	
<code><main class="main"></code>	– flex:1; scrollable; vertical stack of sections

<code>.kpi-strip</code>	- 6-column grid of KPI cards
<code>.section-bar (sticky)</code> chips	- title, date, view toggle, export btn, metric
<code>#view-table</code> active	- table view container; hidden when Cards is
<code>.table-wrap</code>	- overflow-x:auto wrapper for wide table
<code>.hourly-table</code>	- the main data table
<code>#view-cards</code> active	- cards view container; hidden when Table is
<code>.summary-grid</code>	- auto-fill grid of <code>.summary-card</code>
<code>#call-log-section</code>	- only shown when an agent is selected
<code>.section-bar</code>	- call log header + search/filter/export controls
<code>.call-log-wrap</code>	- the call-table + load more button

Key IDs used by JS:

ID	What JS Does With It
kpi-total, kpi-unique, kpi-connected, kpi-talk, kpi-duration, kpi-agents	animateValue() or textContent =
kpi-connect-rate, kpi-sub-date, kpi-sub-agents	textContent = for subtitles
section-title, section-date	textContent = on filter change
date-filter, tl-filter, pool-filter, agent-search	Read by applyFilters() and filterAgentList()
team-list	innerHTML = by renderAgentList()
hourly-table-wrap	innerHTML = by renderBreakdown()
summary-grid	innerHTML = by renderSummaryCards()
call-log-section	style.display toggle
call-thead, call-tbody	innerHTML = and appendChild() by loadCallLog()
call-log-count	textContent =
call-search, call-dir-filter	Read by loadCallLog()
load-more-btn	style.display toggle
last-sync-text	textContent =
countdown-text, countdown-fill	Updated every second by updateCountdownUI()
export-excel-btn, btn-export-breakdown	Button state management during export
btn-table, btn-cards	.active class toggle by setView()
chip-calls, chip-unique, chip-talk, chip-dur	.on class toggle by toggleMetric()

4. Frontend Patterns — Deep Dive

innerHTML vs appendChild — When Each Is Used

`renderBreakdown()`, `renderAgentList()`, `renderSummaryCards()` all use `innerHTML =` on their container. This completely replaces the DOM on every data refresh — simple, predictable, no stale-state bugs.

`loadCallLog()` uses `appendChild(tr)` on `<tbody>` when appending more pages. This avoids wiping existing rows when the user clicks "Load More". When the page resets (`callLogPage === 0`), `tbody.innerHTML = ''` wipes it first.

Rule: Use `innerHTML =` for full re-renders. Use `appendChild` only for incremental appending.

The `esch()` Function — Why It Matters

javascript

```
function esch(s) {  
  return String(s || '').replace(/&/g, '&amp;').replace(/</g, '&lt;').replace(/>/g, '&gt;');  
}
```

Every piece of user-originated data (agent names, emails, phone numbers, call reasons) goes through `esch()` before being inserted into `innerHTML`. This prevents XSS — if an agent's name contained `<script>alert(1)</script>`, without escaping it would execute.

Note: `"` → `"` is needed because agent emails are used inside `onclick="selectAgent('${esch(...)})'"`. Without quote escaping, an email like `test"@domain.com` would break the HTML attribute.

The `fmtHourRange()` Local Function

Defined inside both `renderBreakdown()` and `exportBreakdownExcel()` (duplicated). It converts an integer hour to a human-readable range:

javascript

```
function fmtHourRange(h) {
  const fmt = hr => {
    if (hr === 0 || hr === 24) return '12 AM';
    if (hr === 12) return '12 PM';
    return hr < 12 ? `${hr} AM` : `${hr - 12} PM`;
  };
  return `${fmt(h)}-${fmt(h+1)}`;
}
// fmtHourRange(9) → "9 AM-10 AM"
// fmtHourRange(12) → "12 PM-1 PM"
// fmtHourRange(0) → "12 AM-1 AM"
```

If you need this elsewhere, extract it to a module-level function.

Debounced Search

```
javascript

let callSearchTimeout = null;

function searchCallLog() {
  clearTimeout(callSearchTimeout);
  callSearchTimeout = setTimeout(() => {
    callLogPage = 0;
    loadCallLog();
  }, 400); // 400ms debounce
}
```

This fires the API call 400ms after the user stops typing, not on every keystroke. The `clearTimeout` + `setTimeout` pattern is the standard debounce implementation. The `callSearchTimeout` variable is module-level (not in `state`) — fine because there's only one search input.

The agent sidebar search (`filterAgentList`) does NOT debounce — it filters the already-loaded `state.allAgents` array client-side, so no API call is needed and it can be instant.

`URLSearchParams` Usage Pattern

All API calls build params the same way:

```
javascript
```

```
const params = new URLSearchParams();
if (state.tl) params.set('tl', state.tl);
if (state.pool) params.set('pool', state.pool);
if (state.activeAgent) params.set('agent', state.activeAgent);
if (state.date) params.set('date', state.date);
fetch(`/api/hourly-breakdown/?${params}`)
```

Only truthy values are included. `URLSearchParams` handles encoding automatically — no need to `encodeURIComponent` manually.

`new Date(ms).toLocaleTimeString()` for Call Times

javascript

```
const start = r.start_time_ms ? new Date(r.start_time_ms).toLocaleTimeString() : '-'
```

`toLocaleTimeString()` uses the **browser's local timezone** — this is correct for Dubai-based users viewing the dashboard. The server stores UTC ms; the browser converts to local time automatically. No manual timezone conversion needed in the frontend.

Warning: If the dashboard is accessed from a different timezone (e.g. India),

`toLocaleTimeString()` will show IST times, not Dubai times. If this matters, use:

javascript

```
const start = r.start_time_ms
  ? new Date(r.start_time_ms).toLocaleTimeString('en-AE', { timeZone: 'Asia/Dubai' })
  : '-';
```

5. SheetJS Export System

SheetJS (`xlsx`) is loaded from CDN at the top of the HTML:

html

```
<script src="https://cdnjs.cloudflare.com/ajax/libs/xlsx/0.18.5/xlsx.full.min.js"></script>
```

It exposes a global `XLSX` object. Two export functions exist:

Call Log Export (`exportCallLogExcel`)

1. Sets the button to loading state: `btn.textContent = '⌚ Fetching...'; btn.disabled = true`
2. Fetches `/api/calls/?limit=5000` with current filters
3. Builds a 2D array (`wsData`) where row 0 is headers
4. Uses `XLSX.utils.aoa_to_sheet(wsData)` — "array of arrays to sheet"
5. Sets column widths via `ws['!cols']`
6. Appends the sheet to a workbook
7. Calls `XLSX.writeFile(wb, filename)` — triggers browser download

javascript

```
// Column width format
ws['!cols'] = [
  {wch: 20}, // Agent Name - 20 chars wide
  {wch: 28}, // Email - 28 chars
  // ...
];
```

`wch` = "width in characters". SheetJS converts to actual column width units internally.

Breakdown Export (`exportBreakdownExcel`)

Does NOT make an API call — exports `state.breakdownData` directly. This means:

- Export is instant (no network)
- Export reflects current metric filter state (only visible metrics are exported)
- The `fmtHourRange()` function is used to create human-readable column headers

Header structure for a breakdown export:

```
Agent Name | Account | Employee Code | Pool | TL | Vertical | 9 AM-10 AM - Calls |
9 AM-10 AM - Unique Dials | ... | TOTAL - Calls | ...
```

Adding a New Export

javascript

```

async function exportMyDataExcel() {
  const btn = document.getElementById('my-export-btn');
  btn.textContent = '⌚'; btn.disabled = true;

  try {
    const res = await fetch('/api/my-endpoint/?date=' + state.date);
    const data = await res.json();

    const wsData = [['Col A', 'Col B', 'Col C']]; // header row
    data.rows.forEach(r => {
      wsData.push([r.field_a, r.field_b, r.field_c]);
    });

    const wb = XLSX.utils.book_new();
    const ws = XLSX.utils.aoa_to_sheet(wsData);
    ws['!cols'] = [{wch:20}, {wch:20}, {wch:20}];
    XLSX.utils.book_append_sheet(wb, ws, 'Sheet Name');
    XLSX.writeFile(wb, `export_${state.date}.xlsx`);
  } catch(e) {
    alert('Export failed: ' + e.message);
  } finally {
    btn.textContent = '↓ Export'; btn.disabled = false;
  }
}

```

Always use `try/finally` so the button is re-enabled even if the export fails.

6. Phone Number Masking

javascript

```

function maskNumber(num) {
  const s = String(num || '').trim();
  if (s === '02014' || s === '' || s.length < 6) return s;
  const match = s.match(/^(\\+?\\d{1,3})(\\d+)(\\d{2})$/);
  if (match) return match[1] + '****' + match[3];
  return s.slice(0,3) + '****' + s.slice(-2);
}

```

What this does:

- `'u2014'` check: catches a literal `u2014` string (Unicode escape for em-dash `—`) that didn't render properly)
- Regex `^(\\+?\\d{1,3})(\\d+?)(\\d{2})$`: captures country code, middle digits, last 2 digits
- For `+919876543210`: group 1 = `+91`, group 2 = `98765432`, group 3 = `10` → `+91****10`
- Fallback for non-matching numbers: show first 3 + `****` + last 2

To disable masking (e.g. for admin view):

javascript

```
// In the call log row rendering:
<td>${esch(r.customer_number)}</td>    // remove maskNumber() call
```

To add masking control (toggle button):

javascript

```
let maskEnabled = true;

function toggleMask() {
  maskEnabled = !maskEnabled;
  // Re-render the call log
  callLogPage = 0; loadCallLog();
}

// In call log render:
<td>${esch(maskEnabled ? maskNumber(r.customer_number) : r.customer_number)}</td>
```

7. Heatmap Rendering Logic

Each cell in the hourly table gets a heat bar showing relative volume within the current view.

javascript

```
// Find the maximum calls in any single cell across all agents and hours
let maxCalls = 0;
rows.forEach(r => Object.values(r.hourly).forEach(h => {
  if (h.calls > maxCalls) maxCalls = h.calls;
}));

// For each cell:
const pct = maxCalls ? (hd.calls / maxCalls * 100) : 0;

// Three-tier colour coding:
const heatColor = pct > 70 ? 'linear-gradient(90deg,var(--coral),var(--coral2))'
                        : pct > 40 ? 'linear-gradient(90deg,var(--gold),#ffaa44)'
                        : 'linear-gradient(90deg,var(--cyan),var(--cyan2))';

html += `<div class="heat-bar">
  <div class="heat-fill" style="width:${pct}%;background:${heatColor}"></div>
</div>`;
```

Threshold summary:

- 70% of max → coral (red/orange) — peak activity
- 40-70% of max → gold — moderate activity
- < 40% of max → cyan — low activity

To change thresholds:

```
javascript

// More granular 5-tier system:
const heatColor = pct > 80 ? 'var(--coral)'
                        : pct > 60 ? 'var(--coral2)'
                        : pct > 40 ? 'var(--gold)'
                        : pct > 20 ? 'var(--cyan)'
                        : 'var(--dim)';
```

Limitation: `maxCalls` is computed across ALL agents in the current filtered view. If one agent has 30 calls in an hour and others have 5, the others all appear "cold" even if 5 is their personal peak. To use per-agent relative scaling instead:

```
javascript
```

```
// Inside the rows.forEach loop, compute per-agent max:
let agentMax = 0;
Object.values(r.hourly).forEach(h => { if(h.calls > agentMax) agentMax = h.calls; })
// Then use agentMax instead of maxCalls for this agent's row
```

8. KPI Animation System

Number Count-Up (`animateValue`)

javascript

```
function animateValue(id, end) {
  const el = document.getElementById(id);
  if (!el) return;
  const start = parseInt(el.textContent.replace(/\D/g, '')) || 0; // current display
  const duration = 600; // ms
  const startTime = performance.now();

  function step(now) {
    const progress = Math.min((now - startTime) / duration, 1);
    const eased = 1 - Math.pow(1 - progress, 3); // cubic ease-out
    const current = Math.round(start + (end - start) * eased);
    el.textContent = current.toLocaleString();
    if (progress < 1) requestAnimationFrame(step);
  }
  requestAnimationFrame(step);
}
```

This uses `requestAnimationFrame` for smooth 60fps counting. The easing formula $1 - (1 - \text{progress})^3$ gives cubic ease-out: fast start, slow end. This is the correct easing for count-up animations (feels natural, doesn't drag).

The `start` reads the current DOM value — so on refresh, the counter counts from whatever number is currently displayed to the new number, creating a smooth update rather than jumping to 0 and counting up from there.

`parseInt(el.textContent.replace(/\D/g, ''))` strips commas and other non-digits from `toLocaleString()` formatted numbers.

Time-based KPIs (talk time, total duration) are NOT animated — they get `textContent =` directly because `secsToHMS()` returns a string (`"02:14:33"`), not a number.

Two-Phase KPI Loading

The `renderKPIs()` function does two things:

1. **Immediately** computes totals from `breakdownData` and animates them
2. **Asynchronously** calls `/api/stats/` for the "connected" count and "connect rate" (since `connected` requires `talk_duration > 0` — a per-record check not done in breakdown aggregation)

javascript

```
function renderKPIs(data) {  
  // Phase 1: instant from breakdown data  
  animateValue('kpi-total', totalCalls);  
  animateValue('kpi-unique', totalUniq);  
  // ...  
  
  // Phase 2: async from stats API  
  fetch('/api/stats/' + statsParams)  
    .then(r => r.json())  
    .then(s => {  
      animateValue('kpi-connected', s.connected || 0);  
      document.getElementById('kpi-connect-rate').textContent = `${s.connect_rate}%`  
      if (s.total > 0) animateValue('kpi-total', s.total); // may correct phase 1 v  
    });  
}
```

This means "connected" appears slightly after the other KPIs — not a bug, intentional for responsiveness.

9. Django Patterns Used in This Project

`get_or_create` vs `update_or_create`

python

```
# get_or_create: only inserts if not exists. Skips if exists.  
obj, created = CallRecord.objects.get_or_create(id=call_id, defaults=fields)  
  
# update_or_create: inserts if not exists; updates if exists.  
obj, created = CallRecord.objects.update_or_create(id=call_id, defaults=fields)
```

Do not confuse: `update_or_create` with `id=call_id` will update every field in `defaults` on an existing record. For live sync this is correct (re-sync corrects stale data). For backfill this would overwrite manually-corrected historical records — hence `get_or_create`.

Q Objects for Complex Filters

python

```
from django.db.models import Q

# OR filter (search across multiple fields)
qs.filter(
    Q(customer_number__icontains=search) |
    Q(account_name__icontains=search) |
    Q(full_name__icontains=search)
)

# AND with conditional exclusion (not OR)
qs.exclude(end_call_reason__in=['404 Not Found', 'CAN_NOT_MAKE_CALL'])

# Conditional annotation (count only matching rows)
Count('id', filter=Q(talk_duration__gt=0)) # count connected calls
```

`values()` + `annotate()` for Group-By

python

```
# Django equivalent of SQL GROUP BY account_name
agents = (
    CallRecord.objects
    .values('account_name', 'full_name', 'tl', 'pool') # GROUP BY these columns
    .annotate(
        total_calls=Count('id'),
        total_talk=Sum('talk_duration'),
        connected=Count('id', filter=Q(talk_duration__gt=0)),
    )
    .order_by('-total_calls')
)
```

`values()` before `annotate()` = GROUP BY. `annotate()` before `values()` = per-row annotation (different semantics).

`iterator(chunk_size=500)` for Bulk Operations

```
python

# In reenrich_teams.py:
for rec in CallRecord.objects.all().iterator(chunk_size=500):
    ...
```

Without `.iterator()`, Django loads ALL matching records into memory at once. For 100k+ records this causes OOM errors. `.iterator()` uses a server-side cursor and yields chunks.

`bulk_update` for Batch Writes

```
python

CallRecord.objects.bulk_update(
    batch,
    ['full_name', 'employee_code', 'pool', 'tl', 'vertical']
)
```

`bulk_update(objects, fields)` generates a single SQL statement per batch of 500. Without it, each record would need a separate UPDATE — 100x slower. Only updates the listed fields, not the whole record.

`auto_now_add=True` vs `auto_now=True`

```
python

fetched_at = models.DateTimeField(auto_now_add=True) # set ONCE on creation, never
# auto_now=True would update on EVERY save — NOT what we want here
```

Model Properties vs DB Columns

```
python

@property
def queue_duration_secs(self):
    return max((self.duration or 0) - (self.talk_duration or 0), 0)
```

Properties are Python-only — not queryable in Django ORM. You cannot do:

```
python
```

```
CallRecord.objects.filter(queue_duration_secs__gt=30) # ❌ will error
```

For queryable expressions, use `annotate`:

python

```
from django.db.models import ExpressionWrapper, F, IntegerField
CallRecord.objects.annotate(
    queue_secs=ExpressionWrapper(F('duration') - F('talk_duration'), output_field=IntegerField())
).filter(queue_secs__gt=30)
```

10. PostgreSQL Specifics

`iexact` vs `icontains`

python

```
qs.filter(account_name__iexact='Irfan@bluewave.com') # exact match, case-insensitive
qs.filter(account_name__icontains='irfan')           # substring match, case-insensitive
```

`iexact` uses `UPPER()` in SQL — faster than `icontains` which uses `LIKE`. Always use `iexact` for email lookups where you have the full value.

Index Opportunities

Currently no indexes are defined beyond the primary key. As data grows, add these:

python

```
# In models.py, add to Meta:
class Meta:
    ordering = ['-start_time_ms']
    indexes = [
        models.Index(fields=['start_time_ms']), # date range queries
        models.Index(fields=['account_name']),  # agent filter
        models.Index(fields=['tl']),            # TL filter
        models.Index(fields=['end_call_reason']), # reason filter/exclusion
        models.Index(fields=['start_time_ms', 'tl']), # combined date+TL queries
    ]
```

Create a new migration after adding indexes:

```
bash
```

```
python manage.py makemigrations calls -n 'add_indexes'  
python manage.py migrate
```

Checking Query Performance

```
python
```

```
# In Django shell:  
from django.db import connection, reset_queries  
from django.conf import settings  
  
settings.DEBUG = True # enables query logging  
reset_queries()  
  
# Run your view function logic here  
from calls.models import CallRecord  
from django.db.models import Count  
qs = CallRecord.objects.values('tl').annotate(n=Count('id'))  
list(qs) # force evaluation  
  
print(connection.queries)  
# Shows: [{'sql': '...', 'time': '0.003'}, ...]
```

Database Backup

```
bash
```

```
# Full dump  
pg_dump -U admin -d cdr > backup_$(date +%Y%m%d).sql  
  
# Restore  
psql -U admin -d cdr < backup_20260509.sql  
  
# Just the call records table  
pg_dump -U admin -d cdr -t calls_callrecord > calls_only.sql
```

11. Extending the Excel Lookup — Advanced Scenarios

Adding a New Metadata Column

Example: add `shift` (Morning/Evening) to each agent.

Step 1: Add column to Excel Sheet2 — e.g. `Shift` column with values `Morning` or `Evening`.

Step 2: Update `_build_lookup()` in `team_lookup.py`:

```
python

meta = {
    'full_name': _clean(row.get('Full Name', '')),
    'employee_code': _clean(row.get('Employee code', '')),
    'pool': _clean(row.get('Pool', '')),
    'tl': _clean(row.get('TL', '')),
    'vertical': _clean(row.get('Vertical', '')),
    'shift': _clean(row.get('Shift', '')), # NEW
}
```

Step 3: Update `enrich_record()` default return:

```
python

return lookup.get(key, {
    'full_name': '', 'employee_code': '', 'pool': '', 'tl': '', 'vertical': '',
    'shift': '', # NEW — default to empty string
})
```

Step 4: Add `shift` field to `CallRecord` model:

```
python

shift = models.CharField(max_length=32, blank=True, null=True)
```

Step 5: Add to migration, `upsert_records()` defaults, `api_calls()` response, `reenrich_teams.py` update logic.

Handling Multiple Rows Per Agent (Aliases)

Some agents appear multiple times in the Excel (one row per dialer type). The `_build_lookup()` loop processes them all — the last row with the same prefix wins. To intentionally merge multiple rows for one prefix:

```
python
```

```

if key in lookup:
    # Merge: fill blanks from this row
    existing = lookup[key]
    for field in ('full_name', 'employee_code', 'pool', 'tl', 'vertical'):
        if not existing.get(field) and meta.get(field):
            existing[field] = meta[field]
    # Don't replace the whole entry
else:
    lookup[key] = meta

```

This is already done for Sheet2 rows overriding Sheet1 — it fills blanks but doesn't overwrite.

Debugging the Lookup Build

```

python

# Drop into Django shell and trace the build step by step:
import pandas as pd
from calls.team_lookup import TEAM_XLSX_PATH, _pfx, _clean

xf = pd.ExcelFile(TEAM_XLSX_PATH)

# Check Sheet1
df1 = xf.parse('Sheet1')
print(df1.columns.tolist())
print(df1.head(3))

# Check Sheet2 Stringee rows
df2 = xf.parse('Sheet2')
stringee = df2[df2['Dialer'].astype(str).str.strip().str.lower().isin(['stringee', 's
print(f'{len(stringee)} Stringee rows')
for _, row in stringee.iterrows():
    key = _pfx(row.get('Dialer Name', ''))
    print(f"  prefix={key!r}, TL={row.get('TL', '')!r}, Pool={row.get('Pool', '')!r}")

```

12. Adding New Filter Dimensions

Currently filters are: Date, TL, Pool, Agent. To add a new one (e.g. Vertical):

Step 1: Add vertical to the sidebar HTML

html

```
<div class="filter-block">
  <div class="filter-label">Vertical</div>
  <select class="filter-select" id="vertical-filter" onchange="applyFilters()">
    <option value="">All Verticals</option>
  </select>
</div>
```

Step 2: Populate it in `loadTeamList()`

javascript

```
async function loadTeamList() {
  const res = await fetch('/api/team-list/');
  const data = await res.json();

  // Existing TL/Pool population...

  // Add Vertical
  const verticals = [...new Set(data.agents.map(a => a.vertical).filter(Boolean))].sort();
  const vertSel = document.getElementById('vertical-filter');
  verticals.forEach(v => {
    const opt = document.createElement('option');
    opt.value = v; opt.textContent = v;
    vertSel.appendChild(opt);
  });
}
```

Step 3: Read it in `applyFilters()`

javascript

```
function applyFilters() {
  state.tl = document.getElementById('tl-filter').value;
  state.pool = document.getElementById('pool-filter').value;
  state.vertical = document.getElementById('vertical-filter').value; // NEW
  state.date = document.getElementById('date-filter').value;
  state.activeAgent = null;
  loadBreakdown();
}
```

Add `vertical: ''` to the initial `state` object.

Step 4: Pass it to `loadBreakdown()`

javascript

```
async function loadBreakdown() {
  const params = new URLSearchParams();
  if (state.tl)      params.set('tl', state.tl);
  if (state.pool)    params.set('pool', state.pool);
  if (state.vertical) params.set('vertical', state.vertical); // NEW
  // ...
}
```

Step 5: Handle it in `api_hourly_breakdown()` (views.py)

python

```
vertical = request.GET.get('vertical', '').strip()
# ...
if vertical:
    qs = qs.filter(vertical__iexact=vertical)
```

Step 6: Return verticals from `api_team_list()`

python

```
verticals = sorted({a['vertical'] for a in merged if a.get('vertical')}, key=str.lower)
return JsonResponse({'tls': tls, 'pools': pools, 'agents': merged, 'verticals': verticals})
```

13. Building a New Dashboard Page

For a completely separate view (e.g. a weekly summary page at `/weekly/`):

Step 1: Create the view in `views.py`

python

```
def weekly_dashboard(request):
    return render(request, 'calls/weekly.html')
```

Step 2: Add URL in `urls.py`

python

```
path('weekly/', views.weekly_dashboard, name='weekly_dashboard'),
```

Step 3: Create `calls/templates/calls/weekly.html`

Start with the same CSS variables block from `dashboard.html` (copy the `:root` block and keyframes). Add the `{% load static %}` tag at the top.

html

```
<!DOCTYPE html>
<html lang="en">
<head>
  <meta charset="UTF-8"/>
  <title>Weekly Summary</title>
  {% load static %}
  <link href="https://fonts.googleapis.com/css2?family=Space+Grotesk:wght@400;500;600" rel="stylesheet">
  <style>
    /* Copy :root and base styles from dashboard.html */
    /* Add page-specific styles */
  </style>
</head>
<body>
  <!-- Your content -->
  <script>
    // Your JS
  </script>
</body>
</html>
```

Step 4: Add navigation link in `dashboard.html`

In the topbar:

html

```
<a href="/weekly/" style="font-family:var(--mono);font-size:.65rem;color:var(--muted)">
```

Common Pattern for a New API-Driven Page

```
javascript

// Minimal pattern for any new page that shows filtered data
document.addEventListener('DOMContentLoaded', () => {
  const date = new URLSearchParams(location.search).get('date')
    || new Date().toISOString().split('T')[0];
  loadData(date);
});

async function loadData(date) {
  const res = await fetch('/api/my-endpoint/?date=' + date);
  const data = await res.json();
  render(data);
}

function render(data) {
  // Build HTML and inject
}
```

URL params (`?date=2026-05-09`) let users bookmark and share specific views.

14. Security Model — Full Picture

What Is Protected

- The dashboard page (`/`) — requires session or token
- All API endpoints (`/api/*`) — require session (not token)
- Django admin (`/admin/`) — requires Django superuser login (separate from session)
- Static files (`/static/*`) — NOT protected (intentional — CSS/JS have no sensitive data)

What Is NOT Protected (Deliberately)

- Static assets — no sensitive data there
- The Django admin login page itself — it's a public page with its own auth

Session Mechanism

Sessions are stored in the `django_session` database table (Django's default). Session data is signed with `SECRET_KEY` — if you rotate `SECRET_KEY`, all existing sessions are invalidated immediately.

```
bash
```

```
# Clear all sessions (forces everyone to re-authenticate):  
python manage.py clearsessions # removes only expired sessions  
# OR for all sessions:  
psql -U admin -d cdr -c "DELETE FROM django_session;"
```

Access Token Rotation

1. Update `ACCESS_TOKEN` in `.env`
2. Restart Django (`sudo systemctl restart stringeex-web`)
3. Send new URL (`?token=<new_token>`) to users
4. Old sessions remain valid until expiry (14 days). To invalidate immediately, also clear sessions.

Sensitive Data in the Export

The Excel export includes masked phone numbers (`+91****10`). The raw DB has full numbers. If a TL should not see full numbers:

- The masking happens in JavaScript before export — it uses `maskNumber()` from the JS
- The Python `/api/calls/` endpoint returns `r.customer_number` — the full number
- If you need server-side masking, apply `maskNumber` logic in Python `api_calls()` before the JSON response

For role-based access (TLs see only their team, admins see all), extend the middleware to check the session for a role value, then pass it to the view via `request.session.get('role')`.

15. StringeeX API — Full HTTP Contract

Authentication

```
POST https://{portal}.stringeex.com/v1/account  
Content-Type: application/json  
  
{  
  "email": "...",  
  "password": "..."  
}
```

Response:

```
json
{
  "r": 0,
  "portals": [
    {
      "domain": "bluewave2",
      "auth_token": "eyJ...",
      "name": "BlueWave2"
    }
  ]
}
```

`r: 0` = success. Any other value = error (check `message` field).

`domain` in the response is the plain portal name (`bluewave2`), NOT a full domain (`bluewave2.stringeex.com`). The matching in `get_auth_token_for_portal()` uses the plain name.

Call History

```
GET https://{portal}.stringeex.com/v1/call/history
X-STRINGEE-AUTH: {token}
Content-Type: application/json

?limit=1000&page=1&start_time={ms}&end_time={ms}
```

Response:

```
json
```

```

{
  "r": 0,
  "data": {
    "totalCount": 1500,
    "rows": [
      {
        "id": "call-vn-1-...",
        "account_name": "irfan@bluewave.com",
        "direction": 0,
        "start_time": 1746735000000,
        "end_time": 1746735060000,
        "duration": 60,
        "talk_duration": 45,
        "answer_duration": 45.0,
        "end_call_reason": "USER_END_CALL",
        "customer_number": "919876543210",
        "stringee_number": "917971440960"
      }
    ]
  }
}

```

Pagination: `page=1` returns rows 1-1000, `page=2` returns 1001-2000, etc. The code loops until `len(all_rows) >= totalCount`.

Rate limits: Not documented by StringeeX. The code doesn't implement retry logic. If you hit rate limits, add exponential backoff:

```

python

import time
for attempt in range(3):
    try:
        resp = requests.get(url, headers=headers, params=params, timeout=20)
        resp.raise_for_status()
        break
    except requests.RequestException:
        if attempt == 2: raise
        time.sleep(2 ** attempt)

```

16. Performance & Scaling

Current Bottlenecks

1. `api_hourly_breakdown()` — fetches all matching records as Python objects, aggregates in Python. For 10k+ records per day this is fine. For 100k+, move aggregation to SQL.
2. `api_team_list()` **merge loop** — $O(n)$ loop over all DB agents to match prefixes. Fine for < 1000 agents.
3. **Dashboard table re-render** — `innerHTML = bigString` for 50+ agents $\times$ 8+ hours is a significant DOM operation. No virtualization.
4. `reenrich_teams` **on large DB** — `bulk_update` in batches of 500. For 500k records, this takes ~20 seconds. Acceptable as a maintenance operation.

SQL Aggregation Version of `api_hourly_breakdown`

For large datasets, replace the Python loop aggregation with a raw SQL query:

```
python

from django.db import connection

with connection.cursor() as cursor:
    cursor.execute("""
        SELECT
            account_name,
            full_name, tl, pool, employee_code, vertical,
            EXTRACT(HOUR FROM to_timestamp(start_time_ms/1000.0) AT TIME ZONE 'Asia/
            COUNT(*) AS calls,
            COUNT(DISTINCT customer_number) AS unique_dials,
            SUM(duration) AS total_duration,
            SUM(talk_duration) AS total_talk
        FROM calls_callrecord
        WHERE start_time_ms >= %s
            AND start_time_ms <= %s
            AND COALESCE(end_call_reason, '') NOT IN ('404 Not Found', 'CAN_NOT_MAKE_C
        GROUP BY account_name, full_name, tl, pool, employee_code, vertical, hour
        ORDER BY account_name, hour
        """, [day_start_ms, day_end_ms])
    rows = cursor.fetchall()
```

This offloads all aggregation to PostgreSQL — 10-50x faster for large datasets.

Caching API Responses

For endpoints that don't change frequently (e.g. `api_team_list`), add Django's cache:

```
python

from django.views.decorators.cache import cache_page

@cache_page(60 * 5) # cache for 5 minutes
@require_GET
def api_team_list(request):
    ...
```

Don't cache `api_hourly_breakdown` — it's live data that changes every 4 minutes.

Connection Pooling

For multiple gunicorn workers, use `pgBouncer` or Django's `CONN_MAX_AGE`:

```
python

# In settings.py:
DATABASES = {
    'default': {
        # ...
        'CONN_MAX_AGE': 60, # reuse DB connections for 60 seconds
    }
}
```

17. Testing Patterns

Testing the Team Lookup

```
python
```

```

# tests.py (create in calls/)
from django.test import TestCase
from calls.team_lookup import _pfx, _clean, _build_lookup
from unittest.mock import patch, MagicMock
import pandas as pd
import io

class TeamLookupTest(TestCase):
    def test_prefix_extraction(self):
        self.assertEqual(_pfx('irfan@bluewave.com'), 'irfan')
        self.assertEqual(_pfx('IRFAN@bluewave.com'), 'irfan') # lowercase
        self.assertEqual(_pfx('irfan'), 'irfan') # no @
        self.assertEqual(_pfx(''), '')
        self.assertEqual(_pfx(None), '')

    def test_clean(self):
        self.assertEqual(_clean('Irfan'), 'Irfan')
        self.assertEqual(_clean('nan'), '')
        self.assertEqual(_clean('#N/A'), '')
        self.assertEqual(_clean(' '), '')

    def test_enrich_record(self):
        from calls.team_lookup import enrich_record, reload_team_lookup, _CACHE
        import calls.team_lookup as tl
        # Inject a fake cache
        tl._CACHE = {'irfan': {'full_name': 'Irfan Khan', 'tl': 'Rehan', 'pool': 'Me
        result = enrich_record('irfan@bluewave.com')
        self.assertEqual(result['full_name'], 'Irfan Khan')
        self.assertEqual(result['tl'], 'Rehan')
        tl._CACHE = None # cleanup

```

Testing an API Endpoint

python

```

from django.test import TestCase, Client
from calls.models import CallRecord

class APITest(TestCase):
    def setUp(self):
        self.client = Client()
        # Create test data
        CallRecord.objects.create(
            id='test-001',
            account_name='test@bluewave.com',
            direction='outbound',
            start_time_ms=1746735060000, # 2026-05-09 IST morning
            end_time_ms=1746735120000,
            duration=60, talk_duration=45,
            end_call_reason='USER_END_CALL',
            customer_number='919876543210',
            tl='TestTL', pool='TestPool',
            full_name='Test Agent',
        )
        # Authenticate
        session = self.client.session
        session['authenticated'] = True
        session.save()

    def test_hourly_breakdown_returns_data(self):
        resp = self.client.get('/api/hourly-breakdown/?date=2026-05-09')
        self.assertEqual(resp.status_code, 200)
        data = resp.json()
        self.assertIn('rows', data)
        self.assertEqual(len(data['rows']), 1)
        self.assertEqual(data['rows'][0]['account_name'], 'test@bluewave.com')

    def test_404_not_found_excluded(self):
        CallRecord.objects.create(
            id='test-002', account_name='test@bluewave.com',
            direction='outbound', start_time_ms=1746735060000,
            end_time_ms=1746735120000, duration=10, talk_duration=0,
            end_call_reason='404 Not Found', customer_number='919876543211',
        )
        resp = self.client.get('/api/hourly-breakdown/?date=2026-05-09')
        data = resp.json()
        self.assertEqual(data['rows'][0]['totals']['calls'], 1) # only 1, not 2

```

```
def test_auth_required(self):
    client = Client() # no session
    resp = client.get('/api/hourly-breakdown/')
    self.assertEqual(resp.status_code, 403)
```

Run Tests

bash

```
python manage.py test calls
python manage.py test calls.tests.TeamLookupTest.test_prefix_extraction # single test
python manage.py test --verbosity=2 # verbose output
```

18. Reusable Recipes for Similar Projects

These are patterns from this project that apply directly to any Django + live-data dashboard.

Recipe 1: Session-Based Token Auth Middleware

Copy `calls/auth_middleware.py` verbatim. Change `BYPASS_PATHS` for your URL structure. Register in `MIDDLEWARE` after `SessionMiddleware`. Set `ACCESS_TOKEN` in `.env`.

Recipe 2: Multi-Source Data Ingestion with Upsert

The `upsert_records()` pattern — build a `fields` dict from the raw row, call `get_or_create` or `update_or_create` — works for any external API. Key principles:

- Use the external system's ID as Django's primary key (CharField, not AutoField)
- Store timestamps as milliseconds (BigIntegerField) — avoids timezone conversion at storage time
- Enrich with local data at write time (join at ingest, not at query time)
- Use `get_or_create` for historical data, `update_or_create` for live/current data

Recipe 3: In-Memory Excel Lookup Cache

The `team_lookup.py` pattern — module-level `_CACHE`, built lazily, reloadable via API endpoint — works for any reference data that changes occasionally but is read constantly. Rules:

- Key must be a stable, lowercase, normalized identifier
- Always provide a "no match" default with the same keys
- Expose a `/api/reload-xxx/` endpoint so updates don't require restart
- In multi-worker deployments, use Redis or a shared file signal instead of process memory

Recipe 4: Prefix-Based Email Matching

When two systems use the same people with different email domains, match on the local part (before `@`). This is the `_pfx()` function pattern. Limitations:

- Fails if two different people share a prefix (`john@domain1.com`) and (`john@domain2.com`)
- Always log which prefix matched which record to catch collisions

Recipe 5: Timezone-Correct Date Boundaries

```
python

from zoneinfo import ZoneInfo
from datetime import datetime

LOCAL_TZ = ZoneInfo('Asia/Kolkata') # or your timezone

def day_ms_range(date_str):
    """Return UTC millisecond range for a full local-timezone day."""
    d = datetime.strptime(date_str, '%Y-%m-%d')
    start = datetime(d.year, d.month, d.day, 0, 0, 0, tzinfo=LOCAL_TZ)
    end = datetime(d.year, d.month, d.day, 23, 59, 59, tzinfo=LOCAL_TZ)
    return int(start.timestamp() * 1000), int(end.timestamp() * 1000)
```

Use this exact pattern whenever a user selects a date and you need to query UTC timestamps. Never compute day boundaries in UTC directly.

Recipe 6: Chunked Backfill with Day-by-Day Loop

```
python

current = start_dt
while current <= end_dt:
    day_end = current.replace(hour=23, minute=59, second=59)
    rows = fetch_from_api(current, min(day_end, end_dt))
    upsert(rows)
    current += timedelta(days=1)
```

Fetching month-long ranges in one API call often times out or hits pagination limits. Day-by-day with progress output is more reliable and easier to resume if interrupted.

Recipe 7: requestAnimationFrame Count-Up

The `animateValue()` function is self-contained and reusable. Paste it into any project. Always

read the current DOM value as `start` so updates are smooth rather than resetting to zero. Use cubic ease-out ($(1 - (1-t)^3)$) for counting animations.

Recipe 8: Stable One-File Dashboard

This project's dashboard is deliberately a single HTML file. Advantages:

- No build step, no webpack, no node_modules
- Deployable by copying one file
- Entirely readable in one place
- Fast to load (one request for the page, then JS fetches data)

The tradeoff is that the file is long (1460 lines). Manage this with clear `// — SECTION —` comments and a consistent structure: state → init → data loading → rendering → helpers → exports.

Recipe 9: Paginated Append Pattern for Logs

javascript

```
// Module-level (not in state):
let page = 0;
let total = 0;

async function loadLog() {
  const limit = 50;
  const res = await fetch(`/api/log/?limit=${limit}&offset=${page * limit}`);
  const data = await res.json();
  total = data.total;

  if (page === 0) document.getElementById('tbody').innerHTML = '';

  data.rows.forEach(r => {
    const tr = document.createElement('tr');
    tr.innerHTML = `<td>${r.field}</td>`;
    document.getElementById('tbody').appendChild(tr);
  });

  const btn = document.getElementById('load-more');
  btn.style.display = (page + 1) * limit < total ? 'block' : 'none';
}

function loadMore() { page++; loadLog(); }
function reset() { page = 0; loadLog(); }
```

Recipe 10: SheetJS `aoa_to_sheet` with Column Widths

javascript

```
const wsData = [
  ['Header 1', 'Header 2', 'Header 3'], // row 0 = headers
  ['val1', 'val2', 'val3'], // row 1+
];

const ws = XLSX.utils.aoa_to_sheet(wsData);
ws['!cols'] = wsData[0].map((_, i) => ({ wch: i === 0 ? 30 : 16 })); // header col

const wb = XLSX.utils.book_new();
XLSX.utils.book_append_sheet(wb, ws, 'Sheet Name');
XLSX.writeFile(wb, 'download.xlsx');
```

`aoa_to_sheet` (array of arrays) is the simplest SheetJS API. Use `json_to_sheet` only when you have objects and want auto-headers.

End of Extended Developer Notes (Part 2)